

Jusivo — Appwrite Migration Plan

Generated: 2025-08-10 19:32:51 -05:00

Goal

Migrate the entire stack to Appwrite for auth, database, storage, backend functions, and hosting. Remove Cloudflare and Mocha completely.

What changes

- Auth: Use Appwrite Auth (OAuth2 Google, email/password) via Web SDK
- Database: Move from Cloudflare D1 (SQLite) to Appwrite Databases (collections, attributes, indexes, document permissions)
- Storage: Replace R2 with Appwrite Storage buckets
- Backend APIs and jobs: Replace Cloudflare Worker with Appwrite Functions (HTTP endpoints, scheduled jobs, document generation)
- Email: Prefer Appwrite Messaging (Email channel). If inbound parsing is required, keep SendGrid only as inbound to an Appwrite Function
- Hosting: Keep Appwrite Sites for the SPA
- No Cloudflare and no Mocha in the final architecture

Concrete mapping — Auth

- Was: Mocha Users Service + custom cookies
- Now: Appwrite account sessions via OAuth2 (Google) and/or email/password
- Frontend: Update AuthProvider to use `account.createOAuth2Session` / `account.createEmailPasswordSession` and `account.get()`
- Remove Worker endpoints: `/api/oauth...`, `/api/sessions`, `/api/users/me`

Concrete mapping — Database

- Was: Cloudflare D1 SQL migrations
- Now: Appwrite Database "jusivo" with collections: `clients`, `matters`, `documents`, `document_templates`, `document_versions`, `deadlines`, `hearings`, `time_entries`, `invoices`, `communications`, `deadline_notes`, `user_profiles`
- Permissions: Staff (Team/Role) access to everything; Clients restricted to their own documents/communications; document-level permissions
- Migrations: One-off script to read D1 and create Appwrite documents with proper links and permissions

Concrete mapping — Storage

- Was: R2 + custom `/api/files/:fileKey` routes
- Now: Appwrite Storage bucket "documents"; use `storage.getFileView` / `storage.getFileDownload` directly
- Remove `r2-routes` and any R2-specific logic

Concrete mapping — Email

- Outbound: Use Appwrite Messaging (Email via SMTP) or keep SendGrid from an Appwrite Function for advanced templating
- Inbound: Configure SendGrid Inbound Parse to call a public Appwrite Function (HTTP) that parses and stores communications; link to deadlines/hearings when applicable

Concrete mapping — Backend routes / jobs

- client-portal-lookup (HTTP): sanitize inputs and return safe client info
- client-portal-data (HTTP): fetch client's matters, documents, messages, invoices, upcoming hearings
- email-inbound (HTTP): parse inbound email payload; write to communications; update deadlines/hearings
- notification-scheduler (Scheduled): send deadline/hearing reminders via Messaging
- document-generator (HTTP/Task): generate DOCX/PDF to Storage and return file IDs

Frontend refactor scope

- Replace fetches to /api/... with Appwrite Database SDK calls where allowed by permissions, or with Function HTTP endpoints for server-side logic
- Update AuthProvider to rely solely on Appwrite account.* (sessions, current user)
- Remove cookie/session logic and Worker auth endpoints
- Keep storage helpers; serve document links via Appwrite file URLs

CI/CD

- Sites: Keep existing GitHub Action to build and deploy dist/client to Appwrite Sites
- Functions: Add Appwrite CLI to CI to deploy functions; define appwrite.json and function directories (Node 20 runtime, HTTP/schedule triggers, env vars)
- Example: `npx appwrite deploy function --all`

Data migration (one-off)

- Export D1 data (SQL/CSV/JSON) or query locally
- Write a Node script using Appwrite Admin API to create documents with proper permissions and IDs
- Validate referential integrity (matter_id, client_id, etc.)

Sequenced steps to execute

- 1) Appwrite Console — Create Database collections/attributes/indexes; set permissions; create Storage bucket "documents"; enable OAuth2 Google (and/or email/password); configure Messaging SMTP; create Teams/Roles for staff and decide client role mapping (e.g., user_profiles)
- 2) Functions — Scaffold client-portal-lookup (HTTP), client-portal-data (HTTP), email-inbound (HTTP), notification-scheduler (CRON), document-generator (HTTP/Task); add required environment variables; implement minimal versions and test
- 3) Frontend — Swap AuthProvider to Appwrite account.*; replace API calls with Appwrite SDK/Functions; remove Worker-specific logic and paths; keep storage helpers
- 4) Remove Cloudflare/Mocha — Delete wrangler config and src/worker/*; remove Hono, @cloudflare/vite-plugin, Mocha packages; prune npm deps
- 5) Data migration — Run the one-off importer and verify data and permissions
- 6) Cutover — Deploy functions and site; update allowed origins in Appwrite; verify OAuth callback; smoke test flows

Next actions (options)

- Draft exact Appwrite Database schema (collections/fields/indexes/permissions) for approval
- Scaffold Appwrite Functions (folders + appwrite.json) and CI deployment step
- Refactor AuthProvider and one representative page (e.g., Clients) to demonstrate the integration pattern